ПРАКТИЧЕСКАЯ РАБОТА № 6
УСЛОВНЫЕ ОПЕРАТОРЫ, ENUM И МАРШРУТИЗАЦИЯ СЛУЖЕБНЫХ ЗАЯВОК В JAVA

ЦЕЛЬ РАБОТЫ

Изучить условные операторы if/else if/else, логические выражения с &&, ||, !, перечисления (enum) ApprovalRoute и RiskLevel, граничные значения порогов и декомпозицию правил в private-методы на прикладной задаче маршрутизации служебных заявок. Освоить строгий приоритет правил R1–R5, стратегию fail-fast, защитные условия (контракты метода) и формирование понятных сообщений об ошибках. Получить работающую программу, которая по набору параметров заявки (сумма, срочность, уровень риска, наличие персональных данных) определяет маршрут согласования.

Применить полученные знания в индивидуальном сквозном проекте: завершить реализацию starter_project с методом ApprovalRouter.route, расширить SelfCheck до 13 unit-проверок и data-driven режима чтения sample.csv, реализовать индивидуальный вариант (часть А – корректировка существующего правила или метода, часть Б – добавление нового public метода с предметной логикой). Связать программный код с алгоритмической моделью (Activity-диаграмма) и объектной моделью (диаграмма классов).

СОДЕРЖАНИЕ

1. ПРЕДМЕТНАЯ ПОСТАНОВКА ЗАДАЧИ 2
  1.1. Постановка задачи 2
  1.2. Точный формат входных данных 3
  1.3. Примеры входных строк 3
  1.4. Семантика параметров и бизнес-контекст 3
2. ЗАДАЧИ РАБОТЫ 4
3. ПОРЯДОК ВЫПОЛНЕНИЯ И ОФОРМЛЕНИЯ РАБОТЫ 5
  3.1. Выполнение работы 5
  3.2. Требования к выполнению 6
4. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 6
  4.1. enum как типизированный набор констант 6
  4.2. Условные операторы if, else if, else 7
  4.3. Логические выражения с &&, ||, ! 8
  4.4. Декомпозиция правил в private-методы 8
  4.5. Чтение Activity-диаграммы как алгоритма 9
  4.6. Обязательные правила R1–R5 9
  4.7. Data-driven тестирование через sample.csv 10
5. ПРАКТИЧЕСКАЯ ЧАСТЬ. РЕАЛИЗАЦИЯ И ДИАГРАММЫ 10
  5.1. Структура учебного проекта 10
  5.2. Каркас ApprovalRouter с TODO-заглушками 11
  5.3. Правила R1–R5 и связь с кодом 11
  5.4. Заготовка Activity-диаграммы 12

5.4.1. Назначение 12
5.4.2. Принципы построения Activity-диаграммы 13
Принцип 1. Уровень абстракции – выше кода, ниже «обработать заявку» 13
Принцип 2. Не по блоку на каждый оператор 14
Принцип 3. Не один общий блок 14
Принцип 4. Чем отличается от текста в TODO? 14
Принцип 5. Что добавлять для варианта 14
5.4.3. Текстовое описание узлов исходной диаграммы 15
5.4.4. Логика потока 15
5.4.5. Сдача диаграмм 15
5.5. Заготовка диаграммы классов 15
5.6. SelfCheck: 13 обязательных unit-проверок 16
6. ИНДИВИДУАЛЬНЫЕ ВАРИАНТЫ ЗАДАНИЙ 17
7. РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ 21
8. ТАБЛИЦА ОЦЕНИВАНИЯ РЕЗУЛЬТАТОВ РАБОТЫ 23
9. ФРАГМЕНТ КОДА ДЛЯ АНАЛИЗА 26
10. КОНТРОЛЬНЫЕ ВОПРОСЫ 27
11. СОДЕРЖАНИЕ ОТЧЁТА 28
11.1. Титульный лист 28
11.2. Разделы отчёта 28
11.3. Требования к оформлению 29
12. ИТОГ 29
ПРИЛОЖЕНИЕ А. РАСШИРЕННЫЙ SAMPLE.CSV 29
ПРИЛОЖЕНИЕ Б. ПРИМЕР ЗАПИСИ ВАРИАНТА В DECISIONS.MD 30
ПРИЛОЖЕНИЕ В. ШАБЛОН ОТЧЁТА 31
ПРИЛОЖЕНИЕ Г. ТРЕБОВАНИЯ К ОТВЕТАМ НА КОНТРОЛЬНЫЕ ВОПРОСЫ 32

1. ПРЕДМЕТНАЯ ПОСТАНОВКА ЗАДАЧИ

Разрабатываемое в практической работе приложение автоматизирует маршрутизацию служебных заявок в корпоративной системе документооборота. Предметная областью выступает фрагмент системы согласования: служебная заявка сотрудника должна быть автоматически направлена на один из пяти маршрутов согласования в зависимости от суммы, срочности, уровня риска и наличия персональных данных. Ключевой акцент работы – корректное применение приоритета правил R1–R5 и разрешение пересечений между ними.

1.1. Постановка задачи

Служебная заявка должна быть автоматически направлена на один из маршрутов согласования. Решение зависит от четырёх параметров: amount – сумма заявки в рублях (long, >= 0); urgent – признак срочности (boolean); risk – уровень риска (RiskLevel: LOW, MEDIUM, HIGH); containsPersonalData – признак наличия персональных данных (boolean). Маршрутов согласования пять: STANDARD (стандартный), FAST_TRACK (ускоренный), FINANCE_REVIEW (финансовый контроль), DIRECTOR_REVIEW (директорский), SECURITY_REVIEW (безопасность). Программа должна выбрать ровно один маршрут по правилам раздела 4.6.

Программа должна:

- принимать параметры заявки через метод route(long amount, boolean urgent, RiskLevel risk, boolean containsPersonalData);
- применять правила R1–R5 в строгом порядке приоритета от наиболее строгого к общему;
- возвращать ровно один элемент enum ApprovalRoute – никогда null;
- при срабатывании первого подходящего правила прекращать дальнейшие проверки (fail-fast);
- защищаться от программистских ошибок: amount < 0 и risk == null → IllegalArgumentException;
- использовать константу FINANCE_LIMIT (а не литерал 500000) в правиле R3;
- загружать контрольные значения из data/sample.csv (формат amount;urgent;risk;personal;expected) и сверять результат с ожиданием.

1.2. Точный формат входных данных

Программа принимает параметры через метод route. Дополнительно параметры загружаются из файла data/sample.csv. Формат файла:

- Разделитель – символ «;» (точка с запятой).
- Первая строка – заголовок: amount;urgent;risk;personal. При чтении пропускается (data-driven формат добавляет колонку expected – см. приложение А).
- Типы значений: amount – целое число (long); urgent – true или false; risk – LOW, MEDIUM или HIGH (регистр важен); personal – true или false.
- Surrounding-пробелы не допускаются. Строка 10000; true; LOW; false (с пробелами) считается ошибочной.
- Trailing-разделители не допускаются. Строка 10000;true;LOW;false; – 5 значений вместо 4, ошибка.

1.3. Примеры входных строк

Строка файла	Статус	Параметры заявки
10000;true;LOW;false	Корректна	amount=10000, urgent=true, risk=LOW, personal=false
500000;false;LOW;false	Корректна	amount=500000, urgent=false, risk=LOW, personal=false (граница R3)
10000;true;HIGH;false	Корректна	amount=10000, urgent=true, risk=HIGH, personal=false
10000;true;LOW;true	Корректна	amount=10000, urgent=true, risk=LOW, personal=true
10000; true; LOW; false	Ошибка	Surrounding-пробелы не допускаются
10000;true;LOW;false;	Ошибка	Trailing-разделитель – 5 значений
10000;true;low;false	Ошибка	Регистр risk важен (нужно LOW, не low)
abc;true;LOW;false	Ошибка	amount не является числом

1.4. Семантика параметров и бизнес-контекст

Параметр	Тип Java	Бизнес-смысл	Применение

amount	long	Сумма заявки в рублях. Должна быть >= 0.	Если >= 500000 – финансовый контроль; если < 500000 и urgent – ускоренный.
urgent	boolean	true – заявка срочная, false – обычная.	Срочные заявки малой суммы идут по FAST_TRACK.
risk	RiskLevel	Уровень риска: LOW, MEDIUM, HIGH.	HIGH активирует директор. MEDIUM нейтрален (не активирует R2).
containsPersonalData	boolean	true – содержит персональные данные.	Personal всегда идёт в SECURITY_REVIEW (высший приоритет R1).

Параметры заявки независимы и могут сочетаться. Например, urgent=true AND risk=HIGH одновременно: в этом случае приоритет отдаётся R2 (HIGH > urgent). Personal + HIGH одновременно: приоритет R1 (personal > HIGH). Все пересечения разрешены таблицей приоритетов R1–R5 в разделе 4.6.

Именно для решения этой прикладной задачи в работе реализуется класс ApprovalRouter с методом route, два enum (ApprovalRoute, RiskLevel), расширение SelfCheck с unit- и data-driven проверками, индивидуальные варианты (часть А изменяет правило или метод route, часть Б добавляет новый public метод), Activity-диаграмма алгоритма маршрутизации и диаграмма классов объектной модели.

2. ЗАДАЧИ РАБОТЫ

В ходе выполнения практической работы необходимо:

- актуализировать знания об условных операторах if/else if/else, логических выражениях с &&, ||, ! и enum в Java;
- освоить строгий приоритет правил маршрутизации R1–R5: personal → HIGH → amount >= FINANCE_LIMIT → urgent → STANDARD;
- реализовать класс ApprovalRouter с методом route(long, boolean, RiskLevel, boolean) без изменения сигнатуры;
- применить 5 обязательных правил R1–R5 и стратегию fail-fast: при срабатывании первого подходящего правила возвращается соответствующий маршрут;
- освоить защитные условия (контракты метода): amount < 0 и risk == null → IllegalArgumentException с понятным сообщением;
- использовать константу FINANCE_LIMIT (public static final long = 500_000L) в правиле R3, не литерал;
- расширить SelfCheck до 13 unit-проверок с распределением по пересечениям, базовым сценариям и защитным условиям;
- реализовать data-driven режим SelfCheck: чтение data/sample.csv, разбор строк формата amount;urgent;risk;personal;expected, прогон всех тест-кейсов без перекомпиляции;
- реализовать интерактивный режим для защиты: чтение параметров из System.in, обработка до ввода exit или пустой строки;

- завершить Activity-диаграмму (блок TODO A1) с конкретными формулировками действий, соблюдая пять принципов уровня детализации;
- завершить диаграмму классов: отразить все public-методы ApprovalRouter, оба enum и SelfCheck;
- выбрать индивидуальный вариант и реализовать обе части: А – корректировка существующего правила или метода, Б – добавление нового public метода с предметной логикой;
- адаптировать SelfCheck и sample.csv под выбранный вариант: добавить минимум 2 проверки для части А и 2 для части Б;
- подготовить файл DECISIONS.md (80–150 строк) с описанием варианта, двух принципиальных решений, ответов на 4 контрольных вопроса и анализа фрагмента кода с нарушением приоритета if;
- зафиксировать изменения в Git с меткой PR6: и разместить публичный репозиторий на GitHub;
- подготовить отчёт в формате .docx или .pdf (5–10 страниц) и защитить работу длительностью 10–15 минут.

3. ПОРЯДОК ВЫПОЛНЕНИЯ И ОФОРМЛЕНИЯ РАБОТЫ

Работа выполняется в существующем проекте starter_project, поставляемом преподавателем. Проект компилируется, но намеренно содержит незавершённые места, помеченные TODO. Создание нового проекта не допускается. Изменения выполняются в существующей пакетной структуре src/ru/edu/pr03/.

3.1. Выполнение работы

Практическая часть выполняется последовательно: изучение теории (раздел 4), реализация метода route, расширение SelfCheck, завершение диаграмм. Последовательность доработки проекта:

- Изучить структуру starter_project и определить место TODO в ApprovalRouter.
- Реализовать метод route(long amount, boolean urgent, RiskLevel risk, boolean containsPersonalData) – единственную точку бизнес-логики маршрутизации по правилам R1–R5.
- Применить стратегию fail-fast: при срабатывании первого правила – return, остальные проверки не выполняются.
- Реализовать защитные условия: amount < 0 и risk == null → IllegalArgumentException с понятным сообщением.
- Использовать константу FINANCE_LIMIT в правиле R3 (а не литерал 500000).
- Расширить SelfCheck до 13 unit-проверок с распределением по сценариям (подраздел 5.6).

- Реализовать data-driven режим: чтение data/sample.csv, разбор строк, прогон тест-кейсов.
- Реализовать интерактивный режим для защиты: чтение параметров из System.in до exit или пустой строки.
- Завершить Activity-диаграмму: блок TODO A1 с конкретными формулировками (подраздел 5.4).
- Завершить диаграмму классов: отразить ApprovalRouter, ApprovalRoute, RiskLevel, SelfCheck (подраздел 5.5).
- Выбрать индивидуальный вариант и реализовать обе части: А (корректировка) и Б (новый метод).
- Адаптировать SelfCheck и sample.csv под вариант: +4 проверки и 3–5 строк данных.
- Подготовить DECISIONS.md, отчёт и защиту (разделы 7, 8, 9, 10).

3.2. Требования к выполнению

- сохранить пакетную структуру src/ru/edu/pr03/ и не изменять сигнатуру метода route (кроме случаев, когда выбранный вариант явно требует – варианты 27, 28, 29);
- не использовать Stream API (java.util.stream), включая Arrays.stream, IntStream, Stream.of;
- не использовать лямбда-выражения и method references (::) в любом контексте;
- не использовать Iterable.forEach, List.forEach для итерации и внешние библиотеки (Apache Commons, Guava);
- разрешены: классические if/else if/else, switch, var, switch expressions, text blocks, утилитные методы Math, String, Long, Files, Scanner;
- не создавать отдельный класс Main/App – main уже есть в SelfCheck;
- обязательно использовать константу FINANCE_LIMIT (а не литерал 500000) в правиле R3;
- обязательно сравнивать enum через == (а не через equals), что типобезопасно для enum;
- фиксировать изменения в Git с меткой PR6: и поддерживать актуальность README.md.

4. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

4.1. enum как типизированный набор констант

enum в Java – это специальный тип для представления конечного набора констант. В работе применяются два enum: ApprovalRoute с пятью значениями (STANDARD, FAST_TRACK, FINANCE_REVIEW, DIRECTOR_REVIEW, SECURITY_REVIEW) и RiskLevel с тремя (LOW, MEDIUM, HIGH). Преимущества enum

перед String или int: типобезопасность – невозможно передать несуществующее значение (например, ApprovalRoute.UNKNOWN не скомпилируется); проверка на этапе компиляции – опечатки выявляются сразу; сравнение через == (а не equals), так как гарантируется единственный экземпляр каждого значения.

```
package ru.edu.pr03;

public enum ApprovalRoute {
    STANDARD, FAST_TRACK, FINANCE_REVIEW, DIRECTOR_REVIEW, SECURITY_REVIEW
}

public enum RiskLevel {
    LOW, MEDIUM, HIGH
}
```

Сравнение enum через == корректно и рекомендуется: if (risk == RiskLevel.HIGH). Использование equals тоже работает, но избыточно – для enum JVM гарантирует единственный экземпляр каждого значения в памяти. Опечатка ApprovalRoute.FINANCE_REVIW (вместо FINANCE_REVIEW) приведёт к ошибке компиляции, что снижает количество runtime-багов.

4.2. Условные операторы if, else if, else

Условные операторы управляют потоком выполнения. В задаче маршрутизации применяется цепочка if / else if / else для проверки правил R1–R5 по приоритету. Порядок проверки критически важен: первым должно идти R1 (personal), затем R2 (HIGH), R3 (FINANCE), R4 (urgent), и только в конце R5 (STANDARD как fallback). Изменение порядка приводит к неверным результатам для пересечений (например, urgent+personal должно уйти в SECURITY_REVIEW по R1, а не в FAST_TRACK по R4).

```
public ApprovalRoute route(long amount, boolean urgent,
                       RiskLevel risk, boolean containsPersonalData) {
    // Защитные условия (контракты)
    if (amount < 0) throw new IllegalArgumentException(
        "amount must be >= 0, got " + amount);
    if (risk == null) throw new IllegalArgumentException("risk is null");

    // R1: персональные данные – высший приоритет
    if (containsPersonalData) return ApprovalRoute.SECURITY_REVIEW;
    // R2: высокий риск
    if (risk == RiskLevel.HIGH) return ApprovalRoute.DIRECTOR_REVIEW;
    // R3: крупная сумма
    if (amount >= FINANCE_LIMIT) return ApprovalRoute.FINANCE_REVIEW;
    // R4: срочная заявка малой суммы
    if (urgent && amount < FINANCE_LIMIT) return ApprovalRoute.FAST_TRACK;
    // R5: стандартный маршрут
    return ApprovalRoute.STANDARD;
}
```

Каждое правило – это if с возвратом. После return остальные проверки не выполняются (fail-fast). Это исключает «проваливание» в неправильную ветку и упрощает чтение кода: проверка одного правила не зависит от других.

4.3. Логические выражения с &&, ||, !

Логические операторы && (И, короткое замыкание), || (ИЛИ, короткое замыкание), ! (НЕ) объединяют простые условия в сложные. В правиле R4 применяется &&: urgent && amount < FINANCE_LIMIT – срабатывает только если оба условия истинны. Оператор && использует короткое замыкание: если urgent == false, правое условие amount < FINANCE_LIMIT не вычисляется – это полезно, если правое условие имеет побочные эффекты или может выбросить исключение (например, obj != null && obj.getValue() > 0).

Оператор || применяется в вариантах, где правило активируется при выполнении хотя бы одного из условий (например, вариант 2: risk == HIGH || risk == MEDIUM). Оператор ! инвертирует условие: !containsPersonalData. Скобки управляют приоритетом: (a || b) && c отличается от a || (b && c).

4.4. Декомпозиция правил в private-методы

Метод route можно реализовать как одну цепочку if (5–7 строк). Однако для читаемости рекомендуется вынести проверки в private-методы с говорящими именами: requiresSecurityReview, requiresDirectorReview, requiresFinanceReview, requiresFastTrack. Тогда route использует эти методы, и его тело становится почти декларативным: читатель видит порядок правил, а не детали условия.

```
private boolean requiresSecurityReview(boolean containsPersonalData) {
    return containsPersonalData;
}

private boolean requiresDirectorReview(RiskLevel risk) {
    return risk == RiskLevel.HIGH;
}

private boolean requiresFinanceReview(long amount) {
    return amount >= FINANCE_LIMIT;
}

private boolean requiresFastTrack(long amount, boolean urgent) {
    return urgent && amount < FINANCE_LIMIT;
}
```

Декомпозиция соответствует принципу единственной ответственности (Single Responsibility): каждый private-метод реализует одно правило. Если вариант (раздел 6) изменяет правило R3 (например, FINANCE_LIMIT = 250000), правка локализована в одном месте – константе, а не размазана по коду. Это также упрощает тестирование: можнounit-тестировать каждый private-метод отдельно (хотя в данной работе они не тестируются напрямую – только через публичный route).

4.5. Чтение Activity-диаграммы как алгоритма

Activity-диаграмма визуализирует алгоритм: последовательность действий и ветвлений. В работе Activity-диаграмма показывает поток маршрутизации: получение параметров заявки → определение маршрута через route → серия проверок R1–R5 по приоритету → возврат маршрута или ошибки. Каждый ромб (decision) на диаграмме соответствует if в коде; каждое действие (action) – return или throw. Соответствие должно быть двусторонним: каждое if в route имеет ребро на диаграмме, и наоборот.

Уровень детализации диаграммы – ключевой методический вопрос: слишком мелкий уровень (по блоку на каждый оператор) превращает диаграмму в копию кода; слишком крупный (один блок «определить маршрут») не информирует. Правильный уровень – одно логически завершённое действие или одно ветвление на блок. Принципы построения диаграммы подробно описаны в подразделе 5.4.2.

4.6. Обязательные правила R1–R5

Приоритет	Правило	Условие	Маршрут
1 (высший)	R1: Безопасность	containsPersonalData == true	SECURITY_REVIEW
2	R2: Высокий риск	risk == HIGH	DIRECTOR_REVIEW
3	R3: Финансовый контроль	amount >= FINANCE_LIMIT (500000)	FINANCE_REVIEW
4	R4: Срочная заявка	urgent == true AND amount < FINANCE_LIMIT	FAST_TRACK
5 (низший)	R5: Стандарт	остальные случаи (fallback)	STANDARD

Правила применяются в строгом порядке приоритета: от 1 (R1) к 5 (R5). При срабатывании первого подходящего правила возвращается соответствующий маршрут, остальные проверки не выполняются (fail-fast). Правила содержат пересечения: например, personal AND urgent – сработает R1 (personal > urgent). Personal AND HIGH – сработает R1 (personal > HIGH). HIGH AND amount >= 500000 – сработает R2 (HIGH > суммы). Urgent AND amount >= 500000 – сработает R3 (сумма > urgent). Все пересечения разрешены таблицей приоритетов.

RiskLevel.MEDIUM – нейтральный: не активирует R2. Если остальные правила не срабатывают, заявка с MEDIUM направляется по общим правилам (R3/R4/R5 в зависимости от суммы и срочности). Например, MEDIUM + amount >= 500000 → FINANCE_REVIEW по R3. Это явно проверяется в SelfCheck (проверка 8).

Защитные условия (контракты метода): amount < 0 → IllegalArgumentException("amount must be >= 0, got " + amount); risk == null → IllegalArgumentException("risk is null"). Это не валидация строкового ввода (она выполняется в SelfCheck при чтении sample.csv), а программные контракты: параметры route передаёт вызывающий код, и они должны быть корректны.

Использование константы FINANCE_LIMIT обязательно в правиле R3. Константа объявлена как public static final long FINANCE_LIMIT = 500_000L. Использо-

вание литерала 500000 в коде route запрещено – это «магическое число». Константа обеспечивает единую точку изменения: при реализации варианта (раздел 6) достаточно изменить значение константы, не трогая логику. Менять имя или тип константы запрещено; менять значение – разрешается в рамках варианта.

4.7. Data-driven тестирование через sample.csv

Файл data/sample.csv содержит контрольные значения для тестирования. Формат каждой строки: amount;urgent;risk;personal;expected. Поле expected – ожидаемый маршрут (STANDARD, FAST_TRACK, FINANCE_REVIEW, DIRECTOR_REVIEW, SECURITY_REVIEW) или ERROR (если ожидается IllegalArgumentException). Data-driven режим SelfCheck читает файл, для каждой строки вызывает route, сравнивает с ожиданием, выводит PASS data/N или FAIL data/N: <причина>.

5. ПРАКТИЧЕСКАЯ ЧАСТЬ. РЕАЛИЗАЦИЯ И ДИАГРАММЫ

Практическая часть связывает теоретические сведения раздела 4 с конкретной реализацией. Студент последовательно: изучает структуру starter_project (5.1); читает каркас ApprovalRouter с TODO-заглушками и подсказками (5.2); реализует метод route по правилам R1–R5 (5.3); сверяет реализацию с заготовкой Activity-диаграммы (5.4) и диаграммы классов (5.5); расширяет SelfCheck до 13 unit-проверок (5.6). Каждой задаче кода соответствует теоретический раздел: enum – 4.1; условные операторы – 4.2; логические выражения – 4.3; декомпозиция – 4.4; чтение Activity-диаграммы – 4.5; правила R1–R5 – 4.6.

5.1. Структура учебного проекта

```
starter_project/
├── README.md
├── run_selfcheck.sh            # Unix-скрипт запуска
├── run_selfcheck.bat           # Windows-скрипт запуска
├── data/
│   └── sample.csv              # Контрольные значения
├── src/
│   └── ru/edu/pr03/
│       ├── ApprovalRoute.java      # enum маршрутов (готов)
│       ├── RiskLevel.java          # enum уровней риска (готов)
│       ├── ApprovalRouter.java     # Класс с методом route (TODO)
│       └── SelfCheck.java          # main и тесты
├── activity_partial.dot        # Заготовка Activity-диаграммы
└── class_partial.dot           # Заготовка диаграммы классов
```

Файлы ApprovalRoute.java и RiskLevel.java уже содержат полную реализацию enum. Изменять их содержимое без необходимости запрещено. Исключение – вариант, который требует добавить новый маршрут или уровень риска (варианты 8, 9, 10, 11, 12, 17, 21, 22, 23). В этом случае студент добавляет новые элементы в enum и отражает это на диаграмме классов.

Класс ApprovalRouter содержит нестатический метод route с TODO. Студент должен реализовать тело метода в соответствии со спецификацией подраздела 5.2.

Почему метод нестатический? В ПР6 метод route намеренно нестатический для совместимости с будущими работами, где ApprovalRouter может получить состояние (конфигурация маршрутов, кэш, статистика). В SelfCheck используйте new ApprovalRouter().route(...) – создание экземпляра на каждый вызов допустимо, класс lightweight.

5.2. Каркас ApprovalRouter с TODO-заглушками

Класс ApprovalRouter содержит метод route с TODO-заглушкой. Ниже приведён каркас класса с TODO-заглушкой. Заглушка содержит: сигнатуру метода; правила R1–R5, которые нужно реализовать; подсказки по реализации со ссылками на теоретические разделы. Студент должен заменить TODO на конкретный код в соответствии со спецификацией. Это каркас с подсказками – а не готовое решение.

```
package ru.edu.pr03;

public class ApprovalRouter {
    public static final long FINANCE_LIMIT = 500_000L;

    // TODO 1: route – выбор маршрута по правилам R1-R5
    //   R1: containsPersonalData → SECURITY_REVIEW
    //   R2: risk == HIGH → DIRECTOR_REVIEW
    //   R3: amount >= FINANCE_LIMIT → FINANCE_REVIEW
    //   R4: urgent && amount < FINANCE_LIMIT → FAST_TRACK
    //   R5: else → STANDARD
    //   Подсказка: порядок if строго по приоритету, сравнение enum через ==
    //   Защитные условия (контракты):
    //     amount < 0 → IAE("amount must be >= 0, got " + amount)
    //     risk == null → IAE("risk is null")
    //   См. теорию: 4.2 условные операторы, 4.3 логические выражения,
    //               4.4 декомпозиция в private-методы, 4.6 правила R1-R5
    public ApprovalRoute route(long amount, boolean urgent,
                               RiskLevel risk, boolean containsPersonalData) {
        // TODO: реализуйте защитные условия + правила R1-R5
        return ApprovalRoute.STANDARD;
    }
}
```

При реализации сверяйтесь с теорией: порядок if строго по приоритету R1→R2→R3→R4→R5 – это 4.2; логическое && в R4 – 4.3; декомпозиция в private-методы (requiresSecurityReview и др.) – 4.4; сравнение enum через == – 4.1; использование FINANCE_LIMIT – 4.6. После реализации каждого TODO сверьтесь с таблицей подраздела 5.3.

5.3. Правила R1–R5 и связь с кодом

Правила R1–R5 (таблица в разделе 4.6) – это контракт класса ApprovalRouter. Каждое правило реализуется в методе route и проверяется unit-тестом. Таблица ниже связывает правила, методы и теоретические разделы:

Правило	Метод	Что реализовать	Теория

R1	route	if (containsPersonalData) return SECURITY_REVIEW	4.2 Условные операторы
R2	route	if (risk == RiskLevel.HIGH) return DIRECTOR_REVIEW	4.1 enum, 4.2 if
R3	route	if (amount >= FINANCE_LIMIT) return FINANCE_REVIEW	4.6 Константа FINANCE_LIMIT
R4	route	if (urgent && amount < FINANCE_LIMIT) return FAST_TRACK	4.3 Логические выражения
R5	route	return STANDARD (fallback)	4.2 Условные операторы
Защита 1	route	if (amount < 0) throw IAE("amount must be >= 0, got " + amount)	4.6 Защитные условия
Защита 2	route	if (risk == null) throw IAE("risk is null")	4.6 Защитные условия
Декомпозиция	private-методы	requiresSecurityReview, requiresDirectorReview, requiresFinanceReview, requiresFastTrack	4.4 Декомпозиция

После реализации каждого TODO сверьтесь с таблицей: проверьте, что правило реализовано в нужном методе, выбрасывается правильное исключение (IAE для контрактов), сообщение содержит контекст. Если правило нарушено – код не пройдёт соответствующую unit-проверку SelfCheck.

5.4. Заготовка Activity-диаграммы

Activity-диаграмма визуализирует алгоритм маршрутизации служебной заявки: последовательность проверок и ветвлений. Узлы – этапы обработки (получение параметров, определение маршрута через route, серия проверок R1–R5 по приоритету, возврат маршрута или ошибки). Рёбра – переходы между этапами. Студент должен завершить диаграмму, добавив конкретные формулировки в блок TODO A1.

5.4.1. Назначение

На рисунке 1 дан каркас алгоритма. Красный блок TODO A1 намеренно оставлен неполным. Студент должен сформулировать действие точнее так, чтобы последовательность ветвлений совпадала с реализацией route. Формулировка – конкретная проверка или преобразование, не общая фраза «обработать заявку».

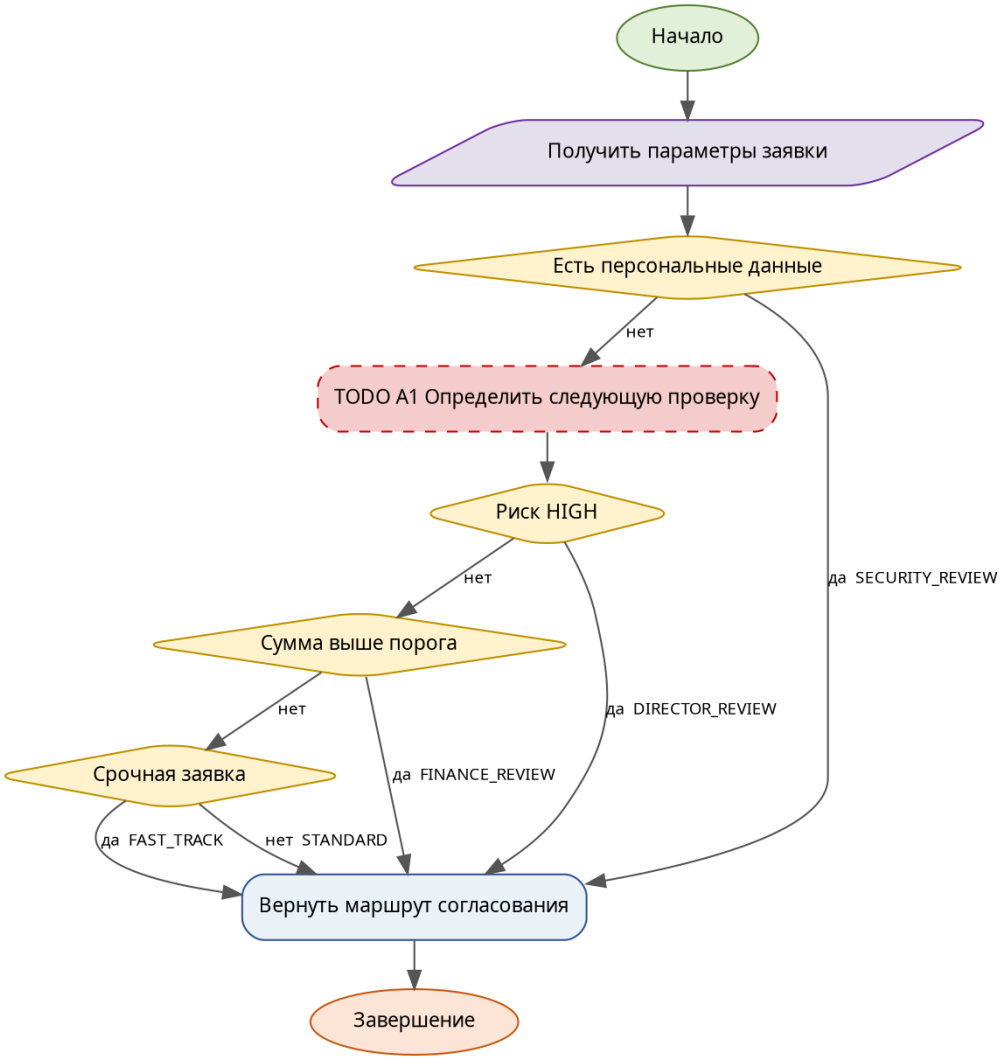


Рисунок 1 – Activity-диаграмма (исходная, с TODO)

5.4.2. Принципы построения Activity-диаграммы

Activity-диаграмма показывает алгоритм маршрутизации – последовательность действий и ветвлений. Правильный уровень детализации – ключевой методический вопрос. Сформулируем принципы:

Принцип 1. Уровень абстракции – выше кода, ниже «обработать заявку»

Каждый блок диаграммы должен описывать одно логически завершённое действие, имеющее смысл как самостоятельный шаг алгоритма. Это не копия кода (где был бы отдельный блок на каждый оператор) и не общая фраза (где был бы один блок «обработать заявку»).

Уровень	Пример блока	Допустимость
Слишком низко (копия кода)	if (risk == RiskLevel.HIGH) return DIRECTOR_REVIEW;	Недопустимо – это код, а не алгоритм
Правильно	«Проверить риск: если HIGH → DIRECTOR_REVIEW»	Допустимо – логический шаг
Слишком высоко	«Определить маршрут»	Недопустимо – не информирует

Принцип 2. Не по блоку на каждый оператор

Если в коде метод route содержит 5 операторов if (по одному на правило), на диаграмме это не 5 отдельных блоков-операторов. Цикл проверки правил R1–R5 – это один-два блока: «Проверить правила R1–R5 по приоритету» (можно разделить на «Проверить R1–R2: безопасность и риск» и «Проверить R3–R4: сумму и срочность», если эти шаги концептуально различны для автора). Каждый блок – это группа операторов, реализующих одно правило: проверка условия + возврат маршрута.

Принцип 3. Не один общий блок

Свести всю обработку к одному блоку «Определить маршрут» – бесполезно. Диаграмма должна показывать ветвления (где принимаются решения) и последовательность существенных шагов. Если в коде есть if/else, на диаграмме должно быть ветвление (ромб). Для маршрутизации это означает: серия проверок R1? → R2? → R3? → R4? → R5 – каждая с возможным возвратом маршрута. Если все проверки не сработали – fallback R5 (STANDARD).

Принцип 4. Чем отличается от текста в TODO?

TODO в исходной диаграмме – это метка незавершённости, указывающая, что блок нужно заполнить. Когда студент завершает блок, он заменяет общую формулировку TODO на конкретное действие, отражающее реализацию. Уровень детализации заполненного блока – тот же, что и у остальных блоков диаграммы (не мельче, не крупнее).

Блок TODO	Недопустимое заполнение	Допустимое заполнение
TODO A1: Проверить правила R1-R5 по приоритету	«if (containsPersonalData) return SECURITY_REVIEW; else if (risk == HIGH) return DIRECTOR_REVIEW; ...»	«Для каждого правила R1-R5: проверить условие, вернуть маршрут если сработало; иначе перейти к следующему правилу; если ни одно не сработало – STANDARD»

Принцип 5. Что добавлять для варианта

Если выбранный вариант (раздел 6) изменяет существующее правило, на диаграмме обновляется формулировка соответствующего блока (например, для варианта 1: «R3: amount >= 250000 вместо 500000»). Если вариант добавляет новое правило (например, COMPLIANCE_REVIEW при personal AND HIGH – вариант 11), добавляется один новый блок (ромб + возврат). Если вариант меняет порядок правил (варианты 13, 14, 15, 16), меняется порядок блоков на диаграмме. Если вариант добавляет новый метод (часть Б), на диаграмме классов добавляется один блок метода – но не на Activity-диаграмме, так как новый метод не входит в основной поток route. Не нужно добавлять блоки на каждый оператор внутри нового метода.

5.4.3. Текстовое описание узлов исходной диаграммы

Activity-диаграмма (activity_partial.dot) содержит следующие узлы (в порядке потока управления):

Узел	Тип	Надпись	Действие студента
start	Начало	«Начало»	Без изменений
n1	Ввод	«Получить параметры заявки»	Без изменений
n2	Действие	«Определить тип доставки через route»	Без изменений
n3	TODO A1	«Проверить правила R1-R5 по приоритету»	Уточнить: для каждого правила – проверить условие, вернуть маршрут если сработало; иначе – STANDARD
n4	Решение	«Маршрут определён?»	Без изменений (защитные условия могут привести к ошибке)
n6	Действие	«Вернуть маршрут»	Без изменений
n7	Действие	«Вернуть ошибку (IAE)»	Без изменений – срабатывает при amount < 0 или risk == null
end	Завершение	«Завершение»	Без изменений

5.4.4. Логика потока

Поток управления: start → n1 → n2 → n3. После n3 → n4 (решение: маршрут определён?). Если «да» (правила R1–R5 сработали корректно) – переход к n6 (вернуть маршрут) → end. Если «нет» (сработало защитное условие amount < 0 или risk == null – выброшено IllegalArgumentException) – переход к n7 (вернуть ошибку) → end.

На защите студент должен показать путь исполнения для конкретного входа. Пример 1: для заявки 10000;true;LOW;true (personal=true): start → n1 → n2 → n3 (R1: containsPersonalData? да) → SECURITY_REVIEW → n4 (да) → n6 → end. Пример 2: для заявки 10000;false;LOW;false (нейтральная): start → n1 → n2 → n3 (R1: нет; R2: нет; R3: нет; R4: нет; R5: STANDARD) → n4 (да) → n6 → end. Пример 3: для заявки -1;true;LOW;false (amount < 0): start → n1 → n2 → n3 (защитное условие сработало) → n4 (нет) → n7 → end.

5.4.5. Сдача диаграмм

Студент сдаёт отредактированные activity.dot. Допускается использование Graphviz DOT или любого UML-редактора (draw.io, PlantUML, Visual Paradigm, dreampuf) с экспортом в PNG.

5.5. Заготовка диаграммы классов

На рисунке 2 – ожидаемая структура решения. Класс ApprovalRouter помечен TODO: студент должен завершить метод route. Если добавляете вспомогательные методы или новые методы (часть Б варианта) – отразите на диаграмме. Оба enum (ApprovalRoute, RiskLevel) показаны как отдельные классы со списком значений.

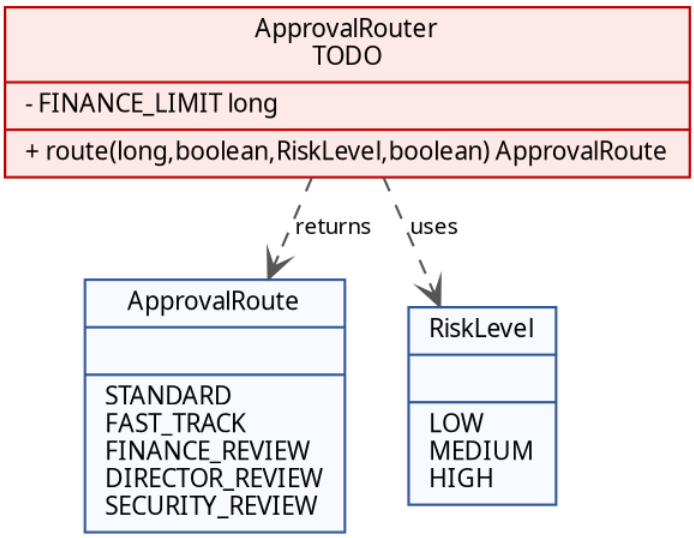


Рисунок 2 – Диаграмма классов (исходная, с TODO)

Связи на диаграмме: ApprovalRouter возвращает ApprovalRoute (стрелка к enum); ApprovalRouter принимает параметр типа RiskLevel (стрелка от enum к ApprovalRouter); SelfCheck вызывает ApprovalRouter (пунктирная стрелка). Поля FINANCE_LIMIT – public static final long, константа. Метод route – нестатический (объяснение в подразделе 5.1).

5.6. SelfCheck: 13 обязательных unit-проверок

SelfCheck – это тестовый раннер: вызывает метод ApprovalRouter.route на заранее подготовленных тестовых случаях и сравнивает результат с ожиданием. Минимальный набор – 13 unit-проверок, распределённых по пересечениям, базовым сценариям и защитным условиям. Каждая проверка соответствует одному правилу или сценарию; это обеспечивает покрытие всех TODO и инвариантов.

№	Категория	Сценарий	Вход (amount;urgent;risk;personal)	Ожидание
1	Пересечения	Personal приоритет над urgent	10000;true;LOW;true	SECURITY_REVIEW
2	Пересечения	HIGH приоритет над urgent	10000;true;HIGH;false	DIRECTOR_REVIEW
3	Пересечения	Personal приоритет над HIGH	10000;false;HIGH;true	SECURITY_REVIEW
4	Пересечения	Сумма приоритет над urgent	600000;true;LOW;false	FINANCE_REVIEW
5	Базовые	FAST_TRACK	10000;true;LOW;false	FAST_TRACK
6	Базовые	FINANCE_REVIEW (граница)	500000;false;LOW;false	FINANCE_REVIEW
7	Базовые	STANDARD	10000;false;LOW;false	STANDARD
8	Базовые	MEDIUM → STANDARD	10000;false;MEDIUM;false	STANDARD
9	Базовые	SECURITY_REVIEW	10000;false;LOW;true	SECURITY_REVIEW
10	Границы	499999 – FAST_TRACK (срочная)	499999;true;LOW;false	FAST_TRACK

№	Категория	Сценарий	Вход (amount;urgent;risk;personal)	Ожидание
11	Границы	500001 – FINANCE_REVIEW	500001;false;LOW;false	FINANCE_REVIEW
12	Защитные	amount < 0	-1;true;LOW;false	IllegalArgumentException
13	Защитные	risk == null	10000;true;null;false	IllegalArgumentException

Data-driven режим: чтение data/sample.csv формата amount;urgent;risk;personal;expected; для каждой строки вызов route; сравнение с ожиданием; PASS data/N или FAIL data/N: <причина>. Интерактивный режим: чтение строк из System.in до exit или пустой строки; вывод «Маршрут: <маршрут>».

После реализации индивидуального варианта (раздел 6) добавляются минимум 4 проверки (2 для части А и 2 для части Б) – итого 17+ PASS. На защите SelfCheck должен выводить только PASS для всех проверок; наличие хотя бы одного FAIL считается регрессией и снижает оценку по критерию «Проверки и устойчивость».

6. ИНДИВИДУАЛЬНЫЕ ВАРИАНТЫ ЗАДАНИЙ

Номер варианта – порядковый номер студента в группе. Каждый вариант состоит из двух частей:

- А – корректировка существующего правила R1–R5 или метода route (изменение должно быть существенным – добавить новую логику или усложнить условие, а не просто поменять константу или знак);
- Б – добавление нового public метода с предметной логикой маршрутизации или анализа заявки в ApprovalRouter.

Сигнатуры менять только если вариант явно требует (варианты 27, 28, 29).

По результатам выполнения задания своего варианта необходимо адаптировать SelfCheck: 13 базовых проверок остаются (с обновлёнными ожиданиями для изменённых правил), добавить минимум 2 проверки для части А и 2 для части Б; обновить sample.csv 3–5 строками с новой логикой. Обновить диаграммы: Activity – формулировки изменённых блоков или 1 новый блок для нового правила; классов – добавить новый метод.

№	Часть А	Часть Б
1	`R3: FINANCE_REVIEW` при `amount >= 250000` (а не 500000). Константу `FINANCE_LIMIT` использовать, изменить значение на 250000.	`addRiskScore(long amount, boolean urgent, RiskLevel risk, boolean containsPersonalData) → int` – вычисляет числовой риск-скор заявки: сумма в рублях / 10000 плюс 100 если `urgent`, плюс 200 если `risk == HIGH`, плюс 50 если `risk == MEDIUM`. Возвращает целое число.
2	`R2: DIRECTOR_REVIEW` при `risk == HIGH OR risk == MEDIUM`. `MEDIUM` больше не нейтральный, активирует директор.	`addRiskWeight(RiskLevel risk) → int` – возвращает вес уровня риска: `LOW` = 0, `MEDIUM` = 50, `HIGH` = 100. Используется для числовой оценки риска в других методах.

№	Часть А	Часть Б
3	R4: FAST_TRACK только если urgent == true AND amount < 100000 (а не 500000). Срочные заявки 100000–499999 → STANDARD.	addUrgencyFlag(long amount, boolean urgent) → boolean – возвращает true, если заявка действительно срочная и может быть ускорена: urgent == true AND amount < 100000. Иначе false. Используется для фильтрации ложных срочных.
4	R5: STANDARD только если amount < 50000 AND risk == LOW AND !urgent. Иначе FINANCE_REVIEW или DIRECTOR_REVIEW.	addStandardEligibility(long amount, boolean urgent, RiskLevel risk, boolean containsPersonalData) → boolean – возвращает true, если заявка подходит под стандартный маршрут: amount < 50000, urgent == false, risk == LOW, containsPersonalData == false. Иначе false.
5	R1: SECURITY_REVIEW если containsPersonalData == true AND amount > 100000. Personal с малой суммой → STANDARD.	addSecurityLevel(long amount, boolean containsPersonalData) → int – возвращает уровень безопасности: 0 если нет персональных данных, 1 если есть и amount <= 100000, 2 если есть и amount > 100000. Чем выше уровень, тем строже контроль.
6	R4: FAST_TRACK при urgent == true независимо от суммы (даже если amount >= 500000). Но не выше personal/HIGH.	addUrgencyPay(long amount, boolean urgent) → double – вычисляет дополнительную оплату за срочность: если urgent == true, возвращает amount × 0.05 (5% надбавка). Иначе 0.0. Используется для расчёта стоимости срочной обработки.
7	R3: FINANCE_REVIEW при amount >= 1000000 (а не 500000). Усиленный порог для крупного контроля.	addAmountCategory(long amount) → String – возвращает текстовую категорию суммы: «малая» если amount < 100000, «средняя» если 100000 <= amount < 500000, «крупная» если 500000 <= amount < 1000000, «крупнейшая» если amount >= 1000000. Используется для классификации.
8	Добавить маршрут URGENT_FINANCE. Если urgent == true AND amount >= FINANCE_LIMIT → URGENT_FINANCE. Приоритет между R3 и R4. Добавить в enum ApprovalRoute.	addUrgentFinancePay(long amount, boolean urgent, double rate) → double – вычисляет оплату срочной крупной заявки: amount × rate × 1.2 (20% надбавка за срочность). Возвращает сумму в рублях. Используется при маршруте URGENT_FINANCE.
9	Добавить маршрут URGENT_DIRECTOR. Если urgent == true AND risk == HIGH → URGENT_DIRECTOR. Приоритет между R2 и R3. Добавить в enum ApprovalRoute.	addUrgentDirectorFlag(boolean urgent, RiskLevel risk) → boolean – возвращает true, если заявка требует особого внимания директора: urgent == true AND risk == HIGH. Иначе false. Используется для маршрута URGENT_DIRECTOR.
10	Добавить уровень CRITICAL. RiskLevel.CRITICAL выше HIGH. CRITICAL → SECURITY_REVIEW (а не DIRECTOR_REVIEW). Добавить в enum RiskLevel.	addCriticalPay(long amount, RiskLevel risk, double rate) → double – вычисляет оплату критической заявки: если risk == CRITICAL, возвращает amount × rate × 2.0 (100% надбавка). Иначе amount × rate. Возвращает сумму в рублях.

№	Часть А	Часть Б
11	Добавить маршрут COMPLIANCE_REVIEW. Если containsPersonalData == true AND risk == HIGH → COMPLIANCE_REVIEW. Приоритет выше R1 и R2. Добавить в enum ApprovalRoute.	addComplianceScore(boolean containsPersonalData, RiskLevel risk) → int – возвращает счёт комплаенса: 0 если нет personal и risk не HIGH, 100 если есть personal ИЛИ risk == HIGH, 200 если есть personal И risk == HIGH. Используется для маршрута COMPLIANCE_REVIEW.
12	Добавить маршрут EXPRESS. Если urgent == true AND amount < 50000 AND risk == LOW → EXPRESS. Приоритет выше R4. Добавить в enum ApprovalRoute.	addExpressEligibility(long amount, boolean urgent, RiskLevel risk, boolean containsPersonalData) → boolean – возвращает true, если заявка подходит для экспресс-маршрута: urgent == true, amount < 50000, risk == LOW, containsPersonalData == false. Иначе false.
13	R3 (FINANCE_REVIEW) проверяется раньше R2 (DIRECTOR_REVIEW). HIGH + сумма >= 500000 → FINANCE_REVIEW. Изменить порядок if.	addPriorityScore(long amount, RiskLevel risk) → int – вычисляет приоритет заявки для сортировки: amount / 100000 плюс 10 если risk == HIGH. Чем выше score, тем выше приоритет обработки. Возвращает целое число.
14	R4 (FAST_TRACK) имеет высший приоритет. urgent == true → FAST_TRACK независимо от personal/HIGH/суммы. Изменить порядок if.	addUrgencyLevel(boolean urgent) → int – возвращает уровень срочности: 0 если urgent == false, 1 если urgent == true. Используется для определения приоритета FAST_TRACK.
15	R2 (HIGH) приоритетнее R1 (personal). personal AND HIGH → DIRECTOR_REVIEW (не SECURITY_REVIEW). Изменить порядок if.	addRiskPriority(RiskLevel risk, boolean containsPersonalData) → ApprovalRoute – возвращает маршрут на основе комбинации риска и personal: если risk == HIGH → DIRECTOR_REVIEW, если containsPersonalData → SECURITY_REVIEW, иначе STANDARD. Используется для демонстрации изменённого приоритета.
16	R3 (FINANCE) приоритетнее R1 (personal). personal AND сумма >= 500000 → FINANCE_REVIEW. Изменить порядок if.	addFinancePriority(long amount, boolean containsPersonalData) → ApprovalRoute – возвращает маршрут: если amount >= FINANCE_LIMIT → FINANCE_REVIEW, если containsPersonalData → SECURITY_REVIEW, иначе STANDARD. Используется для демонстрации изменённого приоритета.
17	Добавить маршрут BOARD_REVIEW. Если amount > 5000000 → BOARD_REVIEW. Приоритет выше R1. Добавить в enum ApprovalRoute.	addBoardPay(long amount, double rate) → double – вычисляет оплату для совета директоров: amount × rate × 1.5 (50% надбавка за уровень согласования). Возвращает сумму в рублях. Используется при маршруте BOARD_REVIEW.
18	Если containsPersonalData == true AND urgent == true → новый COMBINED_REVIEW. Приоритет между R1 и R2. Добавить в enum ApprovalRoute.	addCombinedScore(boolean containsPersonalData, boolean urgent) → int – возвращает счёт комбинированного риска: 0 если нет personal и не urgent, 100 если есть personal ИЛИ urgent, 200 если есть personal И urgent. Используется для маршрута COMBINED_REVIEW.

№	Часть А	Часть Б
19	Если `risk == HIGH AND amount > 1000000 → BOARD_REVIEW (новый enum)`. Обоснование: высокий риск + крупная сумма – совет.	`addBoardEligibility(long amount, RiskLevel risk) → boolean` – возвращает `true`, если заявка требует согласования совета: `amount > 1000000 AND risk == HIGH`. Иначе `false`. Используется для маршрута `BOARD_REVIEW`.
20	Если `urgent == true AND !containsPersonalData AND risk != HIGH AND amount < 50000 → EXPRESS` (новый `enum`). Приоритет выше `R4`.	`addExpressPay(long amount, double rate) → double` – вычисляет оплату экспресс-заявки: `amount × rate × 0.9` (10% скидка за упрощённый маршрут). Возвращает сумму в рублях. Используется при маршруте `EXPRESS`.
21	Если `amount == 0 → FREE` (новый `enum`, бесплатная заявка). Приоритет высший. Обоснование: бесплатные заявки не требуют согласования.	`addFreeEligibility(long amount) → boolean` – возвращает `true`, если заявка бесплатная: `amount == 0`. Иначе `false`. Используется для маршрута `FREE`.
22	Если `containsPersonalData == true AND risk == HIGH → SECURITY_DIRECTOR (новый комбинированный маршрут)`. Обоснование: оба риска одновременно.	`addCombinedRiskScore(boolean containsPersonalData, RiskLevel risk) → int` – возвращает комбинированный риск-скор: 100 если `containsPersonalData`, плюс 100 если `risk == HIGH`. Если оба – 200. Используется для маршрута `SECURITY_DIRECTOR`.
23	Если `amount > 1000000 AND risk != LOW → FULL_REVIEW (новый enum, все виды контроля)`. Обоснование: крупные рискованные – полный контроль.	`addFullReviewEligibility(long amount, RiskLevel risk) → boolean` – возвращает `true`, если заявка требует полного контроля: `amount > 1000000 AND risk != LOW`. Иначе `false`. Используется для маршрута `FULL_REVIEW`.
24	Добавить проверку: если `amount > 10000000 → IllegalArgumentException("amount too large")`. Обоснование: заявки свыше 10 млн не обрабатываются автоматически.	`addAmountValidation(long amount) → boolean` – возвращает `true`, если сумма в допустимом диапазоне: `0 <= amount <= 10000000`. Иначе `false`. Используется для предобработки перед `route`.
25	Если `risk == null`, трактовать как `RiskLevel.LOW (не выбрасывать исключение)`. Обоснование: толерантность к `null` в вызывающем коде.	`addNullSafeRisk(RiskLevel risk) → RiskLevel` – возвращает `risk` если не `null`, иначе `RiskLevel.LOW`. Используется внутри `route` для безопасной обработки `null`.
26	Если `urgent == true AND containsPersonalData == true → IllegalArgumentException`. Обоснование: срочные персональные не допускаются (требуют ручного согласования).	`addUrgentPersonalCheck(boolean urgent, boolean containsPersonalData) → boolean` – возвращает `true`, если комбинация допустима: НЕ (`urgent == true AND containsPersonalData == true`). Иначе `false`. Используется для валидации перед `route`.
27	Добавить параметр `String department` в `route`. Если `department.equals("IT") → FAST_TRACK` при `urgent (даже если сумма >= 500000)`. Обоснование: `IT`-заявки ускоряются.	`addDepartmentPriority(String department, boolean urgent) → int` – возвращает приоритет отдела: 0 если `department == null` или не `IT`, 1 если `department.equals("IT") AND urgent == true`. Используется для ускоренной обработки `IT`-заявок.

№	Часть А	Часть Б
28	Добавить параметр `boolean recurring` в `route`. Если `recurring == true → STANDARD` (повторная заявка не требует согласования). Обоснование: повторные заявки уже согласованы ранее.	`addRecurringFlag(boolean recurring) → boolean` – возвращает `true`, если заявка повторная и не требует согласования: `recurring == true`. Иначе `false`. Используется для упрощённого маршрута повторных заявок.
29	Добавить `enum ContractType {STANDARD, GOVERNMENT, INTERNATIONAL}`. Если `GOVERNMENT` → всегда `SECURITY_REVIEW`. Если `INTERNATIONAL` → всегда `DIRECTOR_REVIEW`. Обоснование: тип контракта определяет маршрут.	`addContractTypePriority(ContractType type) → ApprovalRoute` – возвращает маршрут на основе типа контракта: `SECURITY_REVIEW` если `GOVERNMENT`, `DIRECTOR_REVIEW` если `INTERNATIONAL`, `null` если `STANDARD` (использовать стандартные правила). Используется перед основными правилами `R1-R5`.
30	Если все параметры «нейтральные» (`amount=0`, `urgent=false`, `risk=LOW`, `personal=false`) → новый маршрут `DEFAULT (а не STANDARD)`. Обоснование: явно отметить дефолт.	`addDefaultDetection(long amount, boolean urgent, RiskLevel risk, boolean containsPersonalData) → boolean` – возвращает `true`, если все параметры нейтральные: `amount == 0`, `urgent == false`, `risk == LOW`, `containsPersonalData == false`. Иначе `false`. Используется для маршрута `DEFAULT`.

7. РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ

По итогам выполнения проект должен:

- компилироваться без ошибок;
- проходить все 13 unit-проверок SelfCheck и минимум 4 проверки варианта (17+ PASS);
- корректно определять маршрут заявки по правилам R1–R5 и разрешать все пересечения;
- выбрасывать IllegalArgumentException с понятным сообщением при amount < 0 и risk == null;
- data-driven режим обрабатывает все строки sample.csv и выводит PASS для каждой;
- интерактивный режим принимает строки из System.in до ввода exit или пустой строки;
- Activity-диаграмма и диаграмма классов завершены и согласованы с кодом;
- индивидуальный вариант реализован в полном объёме (часть А + часть Б);
- репозиторий GitHub публичный и содержит все обязательные файлы.

Структура репозитория:

```
pr6-<фамилия>-<инициалы>/
├── README.md                    # Инструкция сборки и запуска
├── .gitignore                   # Исключает out/, .idea/, *.iml, *.class
├── src/
│   └── ru/
│       └── edu/
│           └── pr03/
```

```
│                   ├── ApprovalRoute.java
│                   ├── RiskLevel.java
│                   ├── ApprovalRouter.java
│                   └── SelfCheck.java
├── data/
│   └── sample.csv
├── diagrams/
│   ├── activity.dot
│   ├── activity.png
│   ├── class.dot
│   └── class.png
└── DECISIONS.md
```

По завершении практической работы № 6 студент должен иметь следующие результаты. Все элементы обязательны – отсутствие любого из них снижает оценку.

№	Элемент	Что именно должно быть готово
1	Работающий проект starter_project	Компилируется командой javac -encoding UTF-8 без ошибок. Метод ApprovalRouter.route реализован по правилам R1–R5 с защитными условиями. SelfCheck расширен до 17+ проверок (13 базовых + минимум 4 проверки варианта, все PASS). Data-driven режим читает data/sample.csv и выводит PASS для каждой строки. Интерактивный режим принимает строки из System.in до ввода exit.
2	Файл DECISIONS.md	Markdown-файл (80–150 строк) в корне репозитория. Содержит: номер варианта и описание частей А и Б; разделы «Решение 1» и «Решение 2» с формулировкой, альтернативой и обоснованием; раздел «Ответы на контрольные вопросы» – 4 ответа по 50–150 слов каждый; раздел «Фрагмент кода для анализа» – 2–4 предложения о нарушении приоритета if.
3	Завершённая Activity-диаграмма	Файлы diagrams/activity.dot и diagrams/activity.png (разрешение >=1200x800 px). Блок TODO A1 заполнен конкретными формулировками действий. Последовательность ветвлений совпадает с фактической реализацией кода route.
4	Завершённая диаграмма классов	Файлы diagrams/class.dot и diagrams/class.png (разрешение >=1200x800 px). Отражает 4 класса: ApprovalRouter, ApprovalRoute, RiskLevel, SelfCheck. Включая новый метод из части Б варианта. Несогласованности между диаграммой и кодом не допускаются.
5	Расширенный sample.csv	Файл data/sample.csv в формате amount;urgent;risk;personal;expected. Содержит минимум 13 строк: корректные заявки, ошибки формата, защитные условия (amount<0, risk=null), граничные значения 499999/500000/500001. Дополнен 3–5 строками с логикой индивидуального варианта.
6	Отчёт REPORT.docx (или .pdf)	Файл в корне репозитория, 5–10 страниц. Times New Roman 12 pt, межстрочный 1.5, поля: верхнее/нижнее 2 см, левое 3 см, правое 1.5 см. Содержит 9 разделов: цель работы, описание реализации route, индивидуальный вариант, Activity-диаграмма (PNG), диаграмма классов (PNG), результаты SelfCheck (скриншот), фрагмент кода для анализа, ответы на контрольные вопросы, ссылка на GitHub.
7	Публичный репозиторий GitHub	Репозиторий pr6-<фамилия>-<инициалы> с публичным доступом (без авторизации). Содержит: README.md с командой сборки; .gitignore (исключает out/, .idea/, *.iml, *.class); src/ru/edu/pr03/ с ApprovalRoute.java, RiskLevel.java, ApprovalRouter.java, SelfCheck.java; data/sample.csv; diagrams/ с DOT и PNG; DECISIONS.md; REPORT.docx/pdf. Все коммиты – от студента. Коммиты начинаются с метки PR6:.

№	Элемент	Что именно должно быть готово
8	Публичная ссылка на отчёт в СДО	Указывается в качестве ответа на задание в СДО. Ведёт на отчёт на облачном ресурсе (Яндекс.Диск, Google Drive) или непосредственно на REPORT.docx/pdf в репозитории GitHub.

Минимальный состав для допуска к защите: пункты 1, 2, 3, 4, 5, 7. Без любого из них работа не принимается. Пункты 6 и 8 сдаются к моменту защиты.

8. ТАБЛИЦА ОЦЕНИВАНИЯ РЕЗУЛЬТАТОВ РАБОТЫ

Защита проводится в форме демонстрации работоспособности проекта и устного ответа на вопросы (не более 5 минут). За выполнение работы над индивидуальным проектом студент получает до 2,0 баллов максимум. Для оценивания используется таблица из 40 показателей. Каждый показатель – конкретное действие, которое студент должен продемонстрировать в редакторе кода или консоли: открыть файл, показать метод, изменить параметр, запустить проверку, объяснить результат. Опрос проводится выборочно по пунктам таблицы; демонстрация завершается на первом неверном ответе; за студентом закрепляется набранное на этот момент число баллов.

№	Категория	Показатель оценивания	Что должен продемонстрировать и объяснить студент
1	Корректность	Продемонстрируйте выбор маршрута для 10000;true;LOW;false	В редакторе кода: откройте класс ApprovalRouter, покажите метод route. В интерактивном режиме введите предложенные параметры. Объясните, какое правило сработало и почему выбран именно этот маршрут.
2	Корректность	Продемонстрируйте выбор маршрута для 500000;false;LOW;false	В интерактивном режиме введите параметры. Объясните, почему сработало правило R3 и почему значение 500000 включено в FINANCE_REVIEW. Покажите соответствующую строку кода.
3	Корректность	Продемонстрируйте выбор маршрута для 10000;true;HIGH;false	В интерактивном режиме введите параметры. Объясните, почему сработало правило R2 (а не R4, хотя urgent=true). Покажите приоритет правил в коде.
4	Корректность	Продемонстрируйте выбор маршрута для 10000;true;LOW;true	В интерактивном режиме введите параметры. Объясните, почему сработало правило R1 (высший приоритет) и почему personal перекрывает urgent.
5	Корректность	Продемонстрируйте обработку personal+HIGH (R1 выше R2)	В интерактивном режиме введите заявку с personal=true и risk=HIGH. Объясните, какой маршрут выбран и почему R1 приоритетнее R2. Покажите порядок if в коде.
6	Корректность	Продемонстрируйте обработку HIGH+сумма≥500000 (R2 выше R3)	В интерактивном режиме введите заявку с risk=HIGH и amount=600000. Объясните, какой маршрут выбран и почему R2 приоритетнее R3. Покажите порядок if в коде.
7	Корректность	Продемонстрируйте обработку срочная+сумма≥500000 (R3 выше R4)	В интерактивном режиме введите заявку с urgent=true и amount=600000. Объясните, какой маршрут выбран и почему R3 приоритетнее R4. Покажите порядок if в коде.

№	Категория	Показатель оценивания	Что должен продемонстрировать и объяснить студент
8	Корректность	Продемонстрируйте обработку MEDIUM (не активирует R2)	В интерактивном режиме введите заявку с risk=MEDIUM. Объясните, почему MEDIUM не активирует правило R2 и какой маршрут выбран вместо DIRECTOR_REVIEW.
9	Корректность	Продемонстрируйте использование константы FINANCE_LIMIT в R3	В редакторе кода: покажите объявление константы FINANCE_LIMIT и её использование в правиле R3. Объясните, почему используется константа, а не литерал 500000.
10	Корректность	Продемонстрируйте обработку amount=0 (свободная заявка)	В интерактивном режиме введите заявку с amount=0. Объясните, какой маршрут выбран и почему 0 входит в допустимый диапазон.
11	Корректность	Продемонстрируйте защитное условие amount<0	В редакторе кода: покажите проверку amount < 0. В интерактивном режиме введите amount=-1. Объясните, какое исключение выбрасывается и какое сообщение содержит.
12	Корректность	Продемонстрируйте защитное условие risk=null	В редакторе кода: покажите проверку risk == null. Объясните, какое исключение выбрасывается. Объясните, почему это защитное условие, а не валидация ввода.
13	Корректность	Продемонстрируйте fail-fast: первое сработавшее правило возвращает маршрут	В редакторе кода: покажите, что после каждого if следует return. Объясните, почему остальные проверки не выполняются после срабатывания первого правила.
14	Корректность	Продемонстрируйте обработку суммы 499999 с urgent=true	В интерактивном режиме введите заявку с amount=499999 и urgent=true. Объясните, почему выбран FAST_TRACK, а не FINANCE_REVIEW. Объясните строгое неравенство в R4.
15	Структура	Продемонстрируйте декомпозицию: private-методы для каждого правила	В редакторе кода: покажите private-методы (requiresSecurityReview и т.д.). Объясните, почему проверки вынесены в отдельные методы и как это упрощает чтение и тестирование.
16	Структура	Продемонстрируйте длину метода route (≤25 строк или декомпозиция)	В редакторе кода: покажите метод route. Объясните, почему метод не длиннее 25 строк. Если используется декомпозиция — объясните, как route вызывает private-методы.
17	Структура	Продемонстрируйте использование именованной константы FINANCE_LIMIT	В редакторе кода: покажите объявление public static final long FINANCE_LIMIT. Объясните, почему константа объявлена именно так и как её изменить для варианта.
18	Структура	Продемонстрируйте отсутствие Stream API и лямбд, объясните запрет	В редакторе кода: покажите, что используются классические if/else. Объясните, почему в данной работе запрещены Stream API и лямбды. Объясните цель запрета.
19	Структура	Продемонстрируйте нестатический метод route, объясните почему	В редакторе кода: покажите, что метод route нестатический (без static). Объясните, почему выбран такой подход и как это связано с будущими работами.
20	Структура	Продемонстрируйте использование enum ApprovalRoute и RiskLevel вместо String	В редакторе кода: покажите enum ApprovalRoute и RiskLevel. Объясните, почему используется enum, а не String или int. Объясните преимущества типобезопасности.

№	Категория	Показатель оценивания	Что должен продемонстрировать и объяснить студент
21	Структура	Продемонстрируйте сравнение enum через ==, а не equals	В редакторе кода: покажите сравнение risk == RiskLevel.HIGH. Объясните, почему для enum используется ==, а не equals. Объясните, что гарантируется единственный экземпляр.
22	Структура	Продемонстрируйте локализацию изменения варианта в одном правиле	В редакторе кода: покажите, какое правило изменено в рамках части А варианта. Объясните, почему изменение затронуло только это правило и как декомпозиция помогает локализовать изменения.
23	Диаграммы	Продемонстрируйте Activity-диаграмму, объясните TODO A1	Откройте activity.png. Покажите блок TODO A1 с заполненной формулировкой. Объясните, какому фрагменту кода соответствует этот блок и почему формулировка соответствует принципам построения.
24	Диаграммы	Продемонстрируйте соответствие Activity-диаграммы коду	Откройте activity.png. Для предложенного преподавателем входа покажите путь исполнения по диаграмме. Объясните, какие блоки соответствуют каким строкам кода.
25	Диаграммы	Продемонстрируйте диаграмму классов с ApprovalRouter, ApprovalRoute, RiskLevel, SelfCheck	Откройте class.png. Покажите все классы и связи между ними. Объясните, почему SelfCheck вызывает ApprovalRouter, и как enum связаны с ApprovalRouter.
26	Диаграммы	Продемонстрируйте новый метод из части Б на диаграмме классов	Откройте class.png. Покажите добавленный метод в ApprovalRouter. Объясните его сигнатуру, назначение и связь с существующими методами.
27	Диаграммы	Продемонстрируйте читаемость PNG (≥1200×800), инструмент	Откройте activity.png и class.png. Покажите, что изображения читаемы. Объясните, какой инструмент использовался для создания диаграмм и как выполнялся экспорт в PNG.
28	Диаграммы	Продемонстрируйте отражение нового маршрута/уровня риска на диаграмме	Если вариант добавляет элемент enum, покажите его на диаграмме классов в ApprovalRoute или RiskLevel. Объясните, как новый маршрут/уровень интегрирован в логику route.
29	Проверки	Продемонстрируйте 13 unit-проверок SelfCheck, все PASS	Запустите SelfCheck. Покажите вывод с количеством прошедших проверок. Объясните, как проверки распределены по правилам и пересечениям. Объясните, какие правила покрывают проверки.
30	Проверки	Продемонстрируйте data-driven режим, прогон sample.csv	Запустите SelfCheck. Покажите вывод data-driven режима. Откройте data/sample.csv и объясните формат строк. Объясните, зачем нужен data-driven режим.
31	Проверки	Продемонстрируйте интерактивный режим System.in	Запустите SelfCheck в интерактивном режиме. Введите два набора параметров. Объясните, для чего используется интерактивный режим и как он применяется на защите.
32	Проверки	Продемонстрируйте проверку исключений с сообщением	В редакторе кода: покажите try/catch в SelfCheck. Объясните, что проверяется помимо типа исключения. Объясните, почему важна проверка текста сообщения.
33	Проверки	Продемонстрируйте +4	Запустите SelfCheck. Покажите дополнительные

№	Категория	Показатель оценивания	Что должен продемонстрировать и объяснить студент
		проверки для варианта (2+2)	проверки варианта. Объясните, какие проверки добавлены для части А и какие — для части Б. Объясните, какие граничные случаи они покрывают.
34	Проверки	Продемонстрируйте повторный SelfCheck после изменения FINANCE_LIMIT, без регрессии	В редакторе кода: измените значение FINANCE_LIMIT. Запустите SelfCheck и покажите результат. Объясните, какие ожидания были обновлены и почему это не регрессия.
35	Защита	Продемонстрируйте путь исполнения для 2 разных вводов по Activity-диаграмме	Откройте activity.png. Для двух предложенных преподавателем входов покажите пути исполнения по диаграмме. Объясните каждый переход и соответствие блоку кода.
36	Защита	Продемонстрируйте объяснение приоритета 2 правил	Объясните, почему одно правило приоритетнее другого (например, R1 выше R2). Покажите порядок if в коде. Объясните, что произойдёт при изменении порядка.
37	Защита	Продемонстрируйте объяснение одного граничного условия	Объясните поведение программы для предложенного преподавателем граничного значения (например, 500000 или 499999). Объясните, почему выбрано нестрогое или строгое неравенство.
38	Защита	Продемонстрируйте внесение изменения FINANCE_LIMIT (500000→250000) за ≤3 мин	В редакторе кода: измените значение константы FINANCE_LIMIT. Обновите ожидания в SelfCheck при необходимости. Запустите SelfCheck и покажите результат. Объясните, почему вынесение порога в константу упрощает изменение.
39	Защита	Продемонстрируйте DECISIONS.md с 4 ответами по 100+ слов	Откройте DECISIONS.md. Покажите четыре ответа на контрольные вопросы. Объясните структуру каждого ответа и какие элементы он содержит.
40	Защита	Продемонстрируйте анализ фрагмента кода (приоритет правил или off-by-one)	В DECISIONS.md покажите раздел с анализом фрагмента кода. Объясните ошибку, её причину и исправление. Объясните связь ошибки с правилами R1–R5.
Итого		40 демонстрируемых действий × 0,05	Демонстрация завершается на первом неверном ответе

Структура и требования к отчёту приведены в разделе 11. Требования к составу сдаваемых файлов и Git-репозиторию – в разделе 7.

9. ФРАГМЕНТ КОДА ДЛЯ АНАЛИЗА

В DECISIONS.md студент должен проанализировать фрагмент кода с нарушением приоритета if. Фрагмент приведён ниже; объясните проблему и покажите исправленный подход в DECISIONS.md в разделе «Фрагмент кода для анализа».

```
// Нарушение приоритета: urgent проверяется раньше personal и HIGH
if (urgent) return FAST_TRACK;
if (containsPersonalData) return SECURITY_REVIEW;
```

```
if (risk == HIGH) return DIRECTOR_REVIEW;
if (amount >= FINANCE_LIMIT) return FINANCE_REVIEW;
return STANDARD;
```

Формат ответа – 2–4 предложения (50–150 слов). Указать:

- техническую ошибку – нарушен приоритет R1 > R2 > R3 > R4 > R5;
- пример неверного результата – для 10000;true;LOW;true (urgent=true, personal=true) код вернёт FAST_TRACK, а должен SECURITY_REVIEW по R1;
- исправленный порядок – personal (R1) → HIGH (R2) → amount >= FINANCE_LIMIT (R3) → urgent && amount < FINANCE_LIMIT (R4) → STANDARD (R5);
- связь с правилами R1–R5 – приоритет обеспечивает корректное разрешение пересечений; первая сработавшая проверка возвращает маршрут, остальные не выполняются (fail-fast).

Этот фрагмент также является предметом проверки на защите: преподаватель может попросить прочитать чужой код с нарушением приоритета if и объяснить нарушение (см. показатель 40 в таблице оценивания). Аналогичный фрагмент с нарушением R3 (использование литерала 500000 вместо FINANCE_LIMIT) может быть предложен для самостоятельного анализа.

10. КОНТРОЛЬНЫЕ ВОПРОСЫ

По результатам выполнения работы студент должен ответить на 4 контрольных вопроса. Ответы разместить в файле DECISIONS.md в разделе «Ответы на контрольные вопросы». Объём каждого ответа – 50–150 слов. Требования к структуре и содержанию ответов приведены в Приложении Г.

Вопрос 1

Какая проверка в этой работе защищает программу от наиболее вероятной ошибки входных данных и где она расположена?

Вопрос 2

Какой метод или класс содержит ключевое бизнес-правило и почему вы разместили его именно там?

Вопрос 3

Как Activity-диаграмма отражает один конкретный участок вашего кода?

Вопрос 4

Какое изменение требования в этой работе должно затронуть минимальное количество классов и почему?

№	Вопрос	Формат ответа
1	Какая проверка защищает программу от наиболее вероятной ошибки входных данных и где она располо-	Название проверки + класс + метод + обоснование, почему эта ошибка наиболее вероятная (например, пользователь часто забывает указать персональные данные,

	жена?	или данные о риске могут быть не переданы).
2	Какой метод или класс содержит ключевое бизнес-правило и почему вы разместили его именно там?	Название метода/класса + какое правило (например, «приоритет R1 > R2 > R3 > R4 > R5») + обоснование размещения (например, «метод route в ApprovalRouter, потому что это единственная точка бизнес-логики маршрутизации»).
3	Как Activity-диаграмма отражает один конкретный участок вашего кода?	Блок на диаграмме (например, «ромб R1: personal?») + метод в коде (например, «if (containsPersonalData) return SECURITY_REVIEW; в route») + описание соответствия.
4	Какое изменение требования должно затронуть минимальное количество классов и почему?	Пример изменения (например, «изменение порога FINANCE_LIMIT с 500000 на 250000») + сколько классов затронуто (одна константа в ApprovalRouter) + обоснование локализации (константа вынесена отдельно, изменение не требует правки логики).

11. СОДЕРЖАНИЕ ОТЧЁТА

Отчёт сдаётся в формате .docx или .pdf. В отчёт студент вставляет ссылку на репозиторий GitHub и заполняет разделы по результатам работы. Отчёт следует разместить на любом доступном облачном ресурсе; в качестве ответа на задание в СДО указать публичную ссылку на отчёт.

11.1. Титульный лист

Титульный лист содержит:

- название работы: «Практическая работа №6. Условные операторы, enum и маршрутизация служебных заявок в Java»;
- ФИО студента, группу, номер зачётной книжки;
- номер варианта (из раздела 6);
- ссылку на репозиторий GitHub;
- дату сдачи.

11.2. Разделы отчёта

№	Раздел	Содержание
1	Цель работы	Краткая формулировка цели из раздела 1.1.
2	Описание реализации	Краткое описание метода route (2–3 предложения): что делает, какие правила применяет, в каком порядке. Без программного кода.
3	Индивидуальный вариант	Номер варианта. Часть А: что изменено в существующем правиле или методе route и почему. Часть Б: какой новый метод добавлен, его назначение и логика.
4	Activity-диаграмма	Изображение итоговой Activity-диаграммы (PNG). Краткое описание (2–3 предложения): какие блоки или переходы изменены или добавлены.
5	Диаграмма классов	Изображение итоговой диаграммы классов (PNG). Краткое описание: какие методы добавлены или изменены.
6	Результаты SelfCheck	Скриншот или текст вывода SelfCheck: все 17+ проверки PASS, data-driven работает. Указать количество проверок.
7	Фрагмент кода для анализа	Ответ на задание раздела 9 (2–4 предложения о нарушении приоритета if).

№	Раздел	Содержание
8	Ответы на контрольные вопросы	4 ответа по 50–150 слов (из раздела 10).
9	Ссылка на репозиторий	Публичная ссылка на GitHub.

11.3. Требования к оформлению

- Формат: .docx или .pdf.
- Шрифт: Times New Roman, 12 pt, межстрочный интервал 1.5.
- Поля: верхнее/нижнее 2 см, левое 3 см, правое 1.5 см.
- Изображения диаграмм – читаемы (>=1200x800 px), с подписями «Рисунок N – ...».
- Объём: 5–10 страниц (без титульного листа и приложений).
- Скриншот вывода SelfCheck – в разделе 6, читаемый.

12. ИТОГ

После выполнения ПР6 учебный проект получает завершённую реализацию метода ApprovalRouter.route по правилам R1–R5, расширенный SelfCheck с 13 unit-проверками и data-driven режимом чтения sample.csv, завершённые Activity-диаграмму и диаграмму классов, согласованные с фактической реализацией.

Параметры заявки → route → R1 (personal) / R2 (HIGH) / R3 (FINANCE) / R4 (urgent) / R5 (STANDARD) → маршрут

Реализован индивидуальный вариант: часть А (корректировка существующего правила или метода route) и часть Б (новый public метод с предметной логикой), локализованные в классе ApprovalRouter. SelfCheck обновлён 4 дополнительными проверками, sample.csv – 3–5 строками с новой логикой.

Таким образом, ПР6 закрепляет средства Java для построения логики с ветвлениями (условные операторы, enum, логические выражения, декомпозиция в private-методы) на прикладной задаче маршрутизации служебных заявок и подготавливает студента к изучению объектно-ориентированных средств языка в последующих работах.

ПРИЛОЖЕНИЕ А. РАСШИРЕННЫЙ SAMPLE.CSV

Полный пример data/sample.csv приведён ниже. Строки, начинающиеся с символа #, – комментарии и игнорируются при чтении. Формат каждой строки: amount;urgent;risk;personal;expected. Поле expected содержит ожидаемый маршрут (STANDARD, FAST_TRACK, FINANCE_REVIEW, DIRECTOR_REVIEW, SECURITY_REVIEW) или ERROR (если ожидается IllegalArgumentException).

```
# Расширенный sample.csv – контрольные значения
# Формат: amount;urgent;risk;personal;expected
amount;urgent;risk;personal;expected
10000;true;LOW;false;FAST_TRACK
```

```
500000;false;LOW;false;FINANCE_REVIEW
500001;false;LOW;false;FINANCE_REVIEW
499999;true;LOW;false;FAST_TRACK
10000;false;LOW;false;STANDARD
10000;false;MEDIUM;false;STANDARD
10000;true;HIGH;false;DIRECTOR_REVIEW
10000;false;LOW;true;SECURITY_REVIEW
10000;true;LOW;true;SECURITY_REVIEW
10000;false;HIGH;true;SECURITY_REVIEW
600000;true;LOW;false;FINANCE_REVIEW
600000;false;HIGH;false;DIRECTOR_REVIEW
10000;true;HIGH;true;SECURITY_REVIEW
0;false;LOW;false;STANDARD
10000000;false;LOW;false;FINANCE_REVIEW
-1;true;LOW;false;ERROR
10000;true;null;false;ERROR
```

При чтении: для каждой строки вызывается new ApprovalRouter().route(amount, urgent, risk, personal); результат сравнивается с expected. Строки с ERROR ожидаем выброс IllegalArgumentException. Студент может расширять sample.csv собственными тест-кейсами для проверки индивидуального варианта (часть А и часть Б). На защите преподаватель использует собственный набор входных данных, поэтому решение, подогнанное только под открытый sample.csv, не считается завершённым.

ПРИЛОЖЕНИЕ Б. ПРИМЕР ЗАПИСИ ВАРИАНТА В DECISIONS.MD

Ниже – пример записи DECISIONS.md. Студент должен заполнить разделы по результатам выполнения своего варианта.

```
# DECISIONS.md

## Вариант

Номер варианта: 7

Часть А: R3 – FINANCE_REVIEW при amount >= 1000000 (а не 500000).
         Константу FINANCE_LIMIT изменить на 1000000.
         Обоснование: усиленный порог для крупного контроля;
         заявки 500000–999999 теперь идут по R4 (urgent) или R5 (STANDARD).

Часть Б: addAmountCategory(long amount) → String – возвращает текстовую
         категорию суммы: «малая» (amount < 100000), «средняя»
         (100000 <= amount < 500000), «крупная» (500000 <= amount < 1000000),
         «крупнейшая» (amount >= 1000000).

## Решение 1

Формулировка: изменить значение константы FINANCE_LIMIT с 500_000L на
1_000_000L. Логика route остаётся без изменений – используется константа.

Альтернатива: заменить литерал 500000 в условии if (amount >= 1000000).
```

```
Обоснование: первый вариант предпочтительнее – единая точка изменения,
соответствует принципу DRY. Второй вариант нарушает правило «использовать
константу FINANCE_LIMIT, а не литерал».

## Решение 2

Формулировка: метод addAmountCategory реализован через цепочку if/else if/else
с граничными условиями 100000, 500000, 1000000.

Альтернатива: использовать switch expression по числовому диапазону
(через вычисление amount / 100000).

Обоснование: первый вариант более читаем для 4 категорий; switch expression
потребовал бы дополнительного вычисления и потерял бы явные границы.

## Ответы на контрольные вопросы

### Вопрос 1
[50–150 слов: название проверки + класс + метод + обоснование вероятности]

### Вопрос 2
[50–150 слов: метод route в ApprovalRouter + правило R1–R5 + обоснование]

### Вопрос 3
[50–150 слов: блок на диаграмме + метод в коде + соответствие]

### Вопрос 4
[50–150 слов: пример изменения + сколько классов затронуто + обоснование]

## Фрагмент кода для анализа

[3–5 предложений о нарушении приоритета if: если urgent проверять раньше
personal, то urgent+personal уйдёт в FAST_TRACK вместо SECURITY_REVIEW.
Исправление – порядок R1→R2→R3→R4→R5.]
```

ПРИЛОЖЕНИЕ В. ШАБЛОН ОТЧЁТА

Ниже – структура отчёта (раздел 11). Скопируйте в .docx или .pdf и заполните.

```
ПРАКТИЧЕСКАЯ РАБОТА №6

Условные операторы, enum и маршрутизация служебных заявок в Java

Студент: _____ Группа: _____

Номер зачётной книжки: _____

Номер варианта: _____

Ссылка на репозиторий GitHub: _____

Дата сдачи: _____

1. ЦЕЛЬ РАБОТЫ
```

```
[Краткая формулировка цели из раздела 1.1]

2. ОПИСАНИЕ РЕАЛИЗАЦИИ
route: [2–3 предложения – что делает, какие правила R1–R5 применяет,
        в каком порядке, какие защитные условия]

3. ИНДИВИДУАЛЬНЫЙ ВАРИАНТ №____
Часть А (корректировка): [что изменено в правиле или методе route и почему]
Часть Б (новый метод): [название метода, назначение, логика]

4. ACTIVITY-ДИАГРАММА
[Изображение PNG]
Описание изменений: [2–3 предложения – какие блоки изменены или добавлены]

5. ДИАГРАММА КЛАССОВ
[Изображение PNG]
Описание изменений: [какие методы добавлены/изменены]

6. РЕЗУЛЬТАТЫ SELFCHECK
[Скриншот или текст вывода SelfCheck]
Количество проверок: ______

7. ФРАГМЕНТ КОДА ДЛЯ АНАЛИЗА
[2–4 предложения о нарушении приоритета if]

8. ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ
Вопрос 1: [50–150 слов]
Вопрос 2: [50–150 слов]
Вопрос 3: [50–150 слов]
Вопрос 4: [50–150 слов]

9. ССЫЛКА НА РЕПОЗИТОРИЙ
https://github.com/_____
```

ПРИЛОЖЕНИЕ Г. ТРЕБОВАНИЯ К ОТВЕТАМ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Студент должен заполнить следующие формулировки в DECISIONS.md (или в отдельном файле REFLECTIONS.md). Объём каждого ответа – 50–150 слов.

Вопрос 1

Какая проверка в этой работе защищает программу от наиболее вероятной ошибки входных данных и где она расположена?

Ожидаемые элементы ответа: (1) название проверки (например, «проверка containsPersonalData» или «проверка risk == null»); (2) точное место в коде – класс, метод, строка; (3) обоснование, почему именно эта ошибка «наиболее вероятная» (например, пользователь часто забывает указать персональные данные, или данные о риске могут быть не переданы).

Вопрос 2

Какой метод или класс содержит ключевое бизнес-правило и почему вы разместили его именно там?

Ожидаемые элементы: (1) название метода/класса; (2) какое правило (например, «приоритет R1 > R2 > R3 > R4 > R5»); (3) обоснование размещения (например, «метод route в ApprovalRouter, потому что это единственная точка бизнес-логики маршрутизации, легко тестировать и переиспользовать»).

Вопрос 3

Как Activity-диаграмма отражает один конкретный участок вашего кода?

Ожидаемые элементы: (1) указание блока на диаграмме (например, «ромб R1: personal?» или «TODO A1»); (2) указание метода в коде (например, «if (containsPersonalData) return SECURITY_REVIEW; в route»); (3) описание соответствия – какие строки кода реализуют этот блок диаграммы.

Вопрос 4

Какое изменение требования в этой работе должно затронуть минимальное количество классов и почему?

Ожидаемые элементы: (1) пример изменения (например, «изменение порога FINANCE_LIMIT с 500000 на 250000»); (2) сколько классов/методов затронуто (например, «один класс ApprovalRouter, меняется константа»); (3) обоснование локализации (например, «порог вынесен в константу FINANCE_LIMIT, изменение константы не требует правки логики route»).